

```
<!DOCTYPE html>
<html lang="bn">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Jubayer Craft - 3D Block Game</title>
  <style>
    body { margin: 0; overflow: hidden; font-family: 'Segoe UI', sans-serif; }
    canvas { display: block; }
    #info {
      position: absolute;
      top: 10px;
      left: 10px;
      color: white;
      background: rgba(0,0,0,0.6);
      padding: 12px 18px;
      border-radius: 10px;
      font-size: 14px;
      pointer-events: none;
      z-index: 100;
      border-left: 4px solid #4CAF50;
    }
    #info h2 { margin: 0 0 5px 0; font-size: 18px; color: #4CAF50; }
    #info p { margin: 2px 0; opacity: 0.9; }
    #crosshair {
      position: absolute;
      top: 50%;
      left: 50%;
      transform: translate(-50%, -50%);
      color: white;
      font-size: 30px;
      font-weight: bold;
      pointer-events: none;
      z-index: 100;
      text-shadow: 0 0 5px black;
    }
    #hotbar {
      position: absolute;
      bottom: 20px;
      left: 50%;
      transform: translateX(-50%);
      display: flex;
      gap: 8px;
      z-index: 100;
    }
```

```

}
.slot {
  width: 55px;
  height: 55px;
  border: 3px solid rgba(255,255,255,0.4);
  border-radius: 8px;
  display: flex;
  justify-content: center;
  align-items: center;
  font-size: 24px;
  cursor: pointer;
  background: rgba(0,0,0,0.4);
  transition: 0.2s;
}
.slot.active {
  border-color: #FFD700;
  transform: scale(1.1);
  box-shadow: 0 0 15px rgba(255, 215, 0, 0.5);
}
.slot:hover { border-color: #fff; }
#instructions {
  position: absolute;
  bottom: 90px;
  left: 50%;
  transform: translateX(-50%);
  color: #ccc;
  background: rgba(0,0,0,0.5);
  padding: 8px 16px;
  border-radius: 20px;
  font-size: 12px;
  z-index: 100;
  text-align: center;
  pointer-events: none;
}
</style>
<script src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js"></script>
</head>
<body>

<div id="info">
  <h2>🔨 Jubayer Craft</h2>
  <p>WASD = চলাফেরা | মাউস = ঘোরা</p>
  <p>বাম ক্লিক = ব্লক ভাঙা | ডান ক্লিক = ব্লক বসানো</p>
</div>

```

```
<div id="crosshair">+</div>
```

```
<div id="hotbar">
```

```
  <div class="slot active" data-block="0" onclick="selectBlock(0, this)"> </div>
```

```
  <div class="slot" data-block="1" onclick="selectBlock(1, this)"> </div>
```

```
  <div class="slot" data-block="2" onclick="selectBlock(2, this)"> </div>
```

```
  <div class="slot" data-block="3" onclick="selectBlock(3, this)"> </div>
```

```
  <div class="slot" data-block="4" onclick="selectBlock(4, this)"> </div>
```

```
</div>
```

```
<div id="instructions">  ডান ক্লিক করে ব্লক বসান | বাম ক্লিক করে ভাঙুন</div>
```

```
<script>
```

```
  const scene = new THREE.Scene();
```

```
  scene.background = new THREE.Color(0x87CEEB);
```

```
  scene.fog = new THREE.Fog(0x87CEEB, 20, 80);
```

```
  const camera = new THREE.PerspectiveCamera(75, window.innerWidth /  
window.innerHeight, 0.1, 1000);
```

```
  camera.position.set(0, 5, 10);
```

```
  const renderer = new THREE.WebGLRenderer({ antialias: true });
```

```
  renderer.setSize(window.innerWidth, window.innerHeight);
```

```
  renderer.shadowMap.enabled = true;
```

```
  renderer.shadowMap.type = THREE.PCFSoftShadowMap;
```

```
  document.body.appendChild(renderer.domElement);
```

```
  const ambientLight = new THREE.AmbientLight(0xffffff, 0.6);
```

```
  scene.add(ambientLight);
```

```
  const directionalLight = new THREE.DirectionalLight(0xffffff, 0.8);
```

```
  directionalLight.position.set(50, 100, 50);
```

```
  directionalLight.castShadow = true;
```

```
  directionalLight.shadow.mapSize.width = 2048;
```

```
  directionalLight.shadow.mapSize.height = 2048;
```

```
  directionalLight.shadow.camera.left = -50;
```

```
  directionalLight.shadow.camera.right = 50;
```

```
  directionalLight.shadow.camera.top = 50;
```

```
  directionalLight.shadow.camera.bottom = -50;
```

```
  scene.add(directionalLight);
```

```
  function createTexture(color) {
```

```
    const canvas = document.createElement('canvas');
```

```

    canvas.width = 64;
    canvas.height = 64;
    const ctx = canvas.getContext('2d');
    ctx.fillStyle = color;
    ctx.fillRect(0, 0, 64, 64);
    ctx.fillStyle = 'rgba(0,0,0,0.1)';
    for (let i = 0; i < 64; i += 8) {
      for (let j = 0; j < 64; j += 8) {
        if ((i + j) % 16 === 0) ctx.fillRect(i, j, 4, 4);
      }
    }
    return new THREE.CanvasTexture(canvas);
  }

```

```

const blockTypes = [
  { name: 'ঘাস', color: '#4CAF50', texture: createTexture('#4CAF50') },
  { name: 'মাটি', color: '#8B4513', texture: createTexture('#8B4513') },
  { name: 'পাথর', color: '#9E9E9E', texture: createTexture('#9E9E9E') },
  { name: 'ইট', color: '#D32F2F', texture: createTexture('#D32F2F') },
  { name: 'পানি', color: '#2196F3', texture: createTexture('#2196F3') }
];

```

```

const blockMaterials = blockTypes.map(bt => new THREE.MeshLambertMaterial({ map:
bt.texture }));
const blocks = {};
const blockSize = 1;

```

```

function getKey(x, y, z) { return `${x},${y},${z}`; }

```

```

function addBlock(x, y, z, typeIndex = 0) {
  const key = getKey(x, y, z);
  if (blocks[key]) return;
  const geometry = new THREE.BoxGeometry(blockSize, blockSize, blockSize);
  const material = blockMaterials[typeIndex] || blockMaterials[0];
  const mesh = new THREE.Mesh(geometry, material);
  mesh.position.set(x, y, z);
  mesh.castShadow = true;
  mesh.receiveShadow = true;
  mesh.userData = { x, y, z, type: typeIndex };
  scene.add(mesh);
  blocks[key] = mesh;
}

```

```

function removeBlock(x, y, z) {

```

```

    const key = getKey(x, y, z);
    if (blocks[key]) {
        scene.remove(blocks[key]);
        blocks[key].geometry.dispose();
        delete blocks[key];
    }
}

const worldSize = 20;
for (let x = -worldSize; x <= worldSize; x++) {
    for (let z = -worldSize; z <= worldSize; z++) {
        addBlock(x, 0, z, 0);
        if (Math.random() > 0.7) addBlock(x, -1, z, 1);
        if (Math.random() > 0.95 && Math.abs(x) > 3 && Math.abs(z) > 3) {
            for (let h = 1; h <= 3; h++) addBlock(x, h, z, 1);
            for (let dx = -1; dx <= 1; dx++) {
                for (let dz = -1; dz <= 1; dz++) {
                    addBlock(x + dx, 4, z + dz, 0);
                }
            }
        }
    }
}

const player = {
    height: 1.8,
    speed: 0.15,
    velocity: new THREE.Vector3(),
    onGround: false,
    keys: {}
};

const keys = {};
document.addEventListener('keydown', (e) => { keys[e.code] = true; });
document.addEventListener('keyup', (e) => { keys[e.code] = false; });

let pitch = 0;
let yaw = 0;
const sensitivity = 0.002;

document.body.addEventListener('click', () => {
    document.body.requestPointerLock();
});

```

```

document.addEventListener('mousemove', (e) => {
  if (document.pointerLockElement === document.body) {
    yaw -= e.movementX * sensitivity;
    pitch -= e.movementY * sensitivity;
    pitch = Math.max(-Math.PI/2 + 0.1, Math.min(Math.PI/2 - 0.1, pitch));
  }
});

```

```

const raycaster = new THREE.Raycaster();
const centerScreen = new THREE.Vector2(0, 0);
let selectedBlockType = 0;

```

```

function selectBlock(index, element) {
  selectedBlockType = index;
  document.querySelectorAll('.slot').forEach(s => s.classList.remove('active'));
  element.classList.add('active');
}

```

```

document.addEventListener('mousedown', (e) => {
  if (document.pointerLockElement !== document.body) return;
  raycaster.setFromCamera(centerScreen, camera);
  const blockMeshes = Object.values(blocks);
  const intersects = raycaster.intersectObjects(blockMeshes);

  if (intersects.length > 0) {
    const hit = intersects[0];
    const block = hit.object;

    if (e.button === 0) {
      removeBlock(block.userData.x, block.userData.y, block.userData.z);
    } else if (e.button === 2) {
      const normal = hit.face.normal;
      const newX = block.userData.x + normal.x;
      const newY = block.userData.y + normal.y;
      const newZ = block.userData.z + normal.z;
      addBlock(newX, newY, newZ, selectedBlockType);
    }
  }
});

```

```

document.addEventListener('contextmenu', (e) => e.preventDefault());

```

```

function checkCollision(x, y, z) {
  const checkPositions = [

```

```

    [Math.floor(x), Math.floor(y - 1), Math.floor(z)],
    [Math.floor(x), Math.floor(y), Math.floor(z)],
];
for (const [bx, by, bz] of checkPositions) {
    if (blocks[getKey(bx, by, bz)]) return true;
}
return false;
}

function updatePlayer() {
    const speed = 0.12;
    const forward = new THREE.Vector3(-Math.sin(yaw), 0, -Math.cos(yaw));
    const right = new THREE.Vector3(Math.cos(yaw), 0, -Math.sin(yaw));

    const moveDir = new THREE.Vector3();
    if (keys['KeyW']) moveDir.add(forward);
    if (keys['KeyS']) moveDir.sub(forward);
    if (keys['KeyA']) moveDir.sub(right);
    if (keys['KeyD']) moveDir.add(right);

    if (moveDir.length() > 0) {
        moveDir.normalize();
        moveDir.multiplyScalar(speed);
        const newX = camera.position.x + moveDir.x;
        if (!checkCollision(newX, camera.position.y, camera.position.z)) camera.position.x =
newX;
        const newZ = camera.position.z + moveDir.z;
        if (!checkCollision(camera.position.x, camera.position.y, newZ)) camera.position.z =
newZ;
    }

    player.velocity.y -= 0.015;
    const newY = camera.position.y + player.velocity.y;

    if (checkCollision(camera.position.x, newY, camera.position.z)) {
        player.velocity.y = 0;
        player.onGround = true;
    } else {
        camera.position.y = newY;
        player.onGround = false;
    }

    if (keys['Space'] && player.onGround) {
        player.velocity.y = 0.25;
    }
}

```

```
    player.onGround = false;
  }

  camera.rotation.order = 'YXZ';
  camera.rotation.y = yaw;
  camera.rotation.x = pitch;
}

function animate() {
  requestAnimationFrame(animate);
  updatePlayer();
  renderer.render(scene, camera);
}

animate();

window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});

for (let x = 2; x <= 5; x++) {
  for (let z = 2; z <= 5; z++) {
    addBlock(x, 1, z, 3);
  }
}
</script>
</body>
</html>
```